EXPLOITS UNIVERSITY

BSc INFORMATION COMMUNICATION AND TECHNOLOGY

BICT 3202 · OBJECT-ORIENTED ANALYSIS AND DESIGN

WEEK 10

Object-Oriented Design

PROGRAMME	BSc Information Communication and Technology
COURSE CODE / YEAR	BICT 3202 · Year 3
LECTURER	Francis Fweta — ICT Lecturer
DELIVERY	2h Lecture · 1h Tutorial · 1h Practical · 10h Independent Learning

Prepared by Francis Fweta · Exploits University · Week 10 of 16

Contents

1. Week 10 Introduction

Week 9 organised **how** the development work is done (the Unified Process). Week 10 now moves into the **design** part of the syllabus: **Object-Oriented Design** — an overview of design, and the combining of the **Object Model**, the **Dynamic Model** and the **Behavioural Model** that we built in Weeks 4–8.

The most important transition in OOAD happens here:

ANALYSIS — "What does the system need?" DESIGN — "How are we going to build it?"

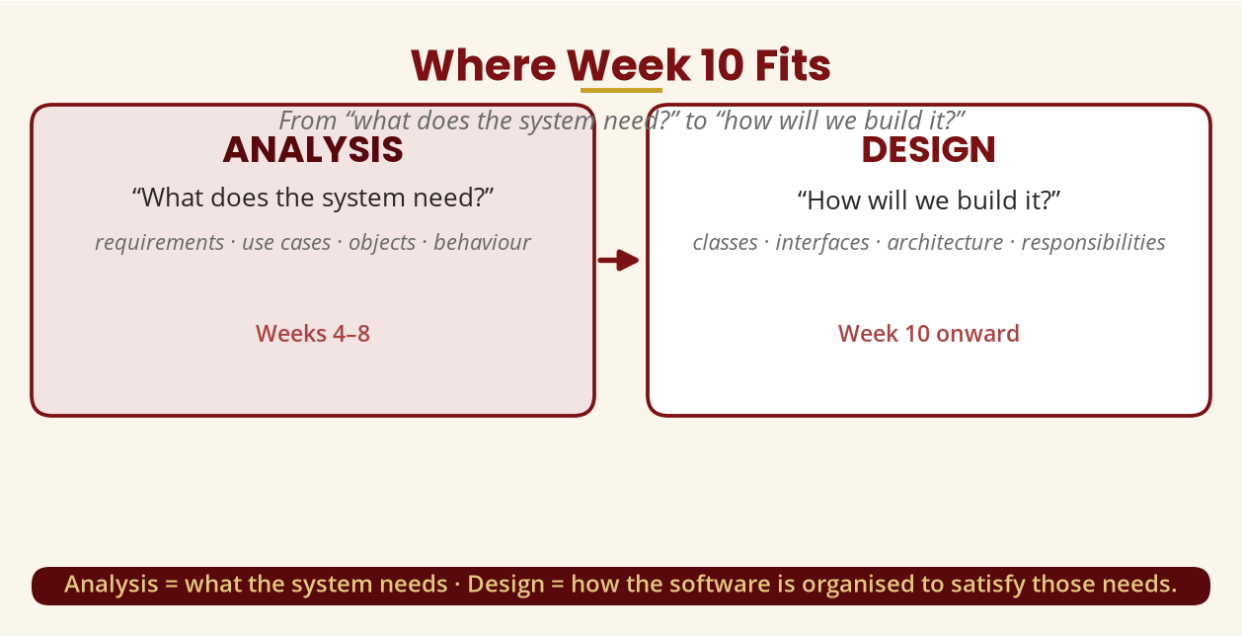

Figure 1 — Where Week 10 fits: analysis design

2. Learning Outcomes

By the end of this week, a student should be able to:

- Define software design and object-oriented design.
- Distinguish object-oriented analysis from object-oriented design, with examples.
- Explain why design is necessary and list the goals of good design.
- Explain responsibility, cohesion, coupling, abstraction, encapsulation and separation of concerns.
- Explain high cohesion and low coupling, and why good design seeks both.
- Describe design architecture, refinement and traceability.
- Combine the Object, Dynamic and Behavioural models into a design suitable for implementation.
- Explain interfaces and, in outline, design patterns and the SOLID principles.

PART A — WHAT IS SOFTWARE DESIGN?

3. The Simple Meaning of Design

Before software, think about constructing a house. A client tells an architect: *"I want a three-bedroom house."* That is a **requirement**. The architect then decides where the bedrooms, kitchen and bathroom go, how electricity is arranged, where pipes run, what materials to use and how the building is supported. That is **design**.

Software is similar. A customer says: *"I want a university student registration system."* That is a requirement. The designer then decides what classes and components are needed, how classes communicate, where business logic belongs, how data is stored, how authentication works, how the system is divided into layers, how errors are handled and how objects collaborate. That is **software design**.

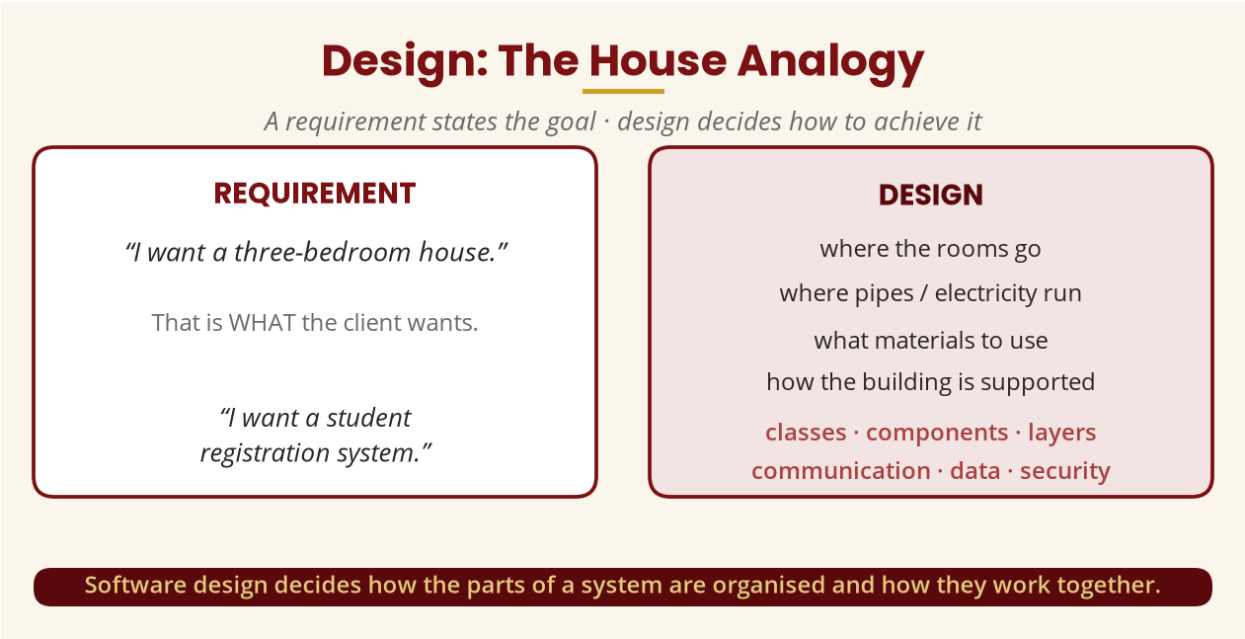

Figure 2 — The house analogy: requirement vs design

4. Definitions

Software design is the process of determining the structure, components, interfaces, interactions and implementation decisions needed to transform requirements and analysis results into a system that can be implemented. A student-friendly version: *"Software design is the process of deciding how a software system will be organised and how its parts will work together to satisfy its requirements."*

Object-Oriented Design (OOD) is the process of designing software around **objects, classes, responsibilities, relationships, interfaces, collaborations, inheritance (where appropriate) and encapsulated data and behaviour**. The designer asks: *"What should the Student object know?"* and *"What should it be responsible for doing?"*

5. Analysis vs Design — THE KEY DISTINCTION

Students often say *"analysis and design are the same thing."* They are related, but not identical. **Analysis** asks **what** the system needs to accomplish — business requirements, user needs, domain concepts, system behaviour, objects, relationships and use cases. **Design** asks **how** the software should be structured and implemented — classes, interfaces, components, architecture, responsibilities, object collaborations, data access, security, persistence and technologies.

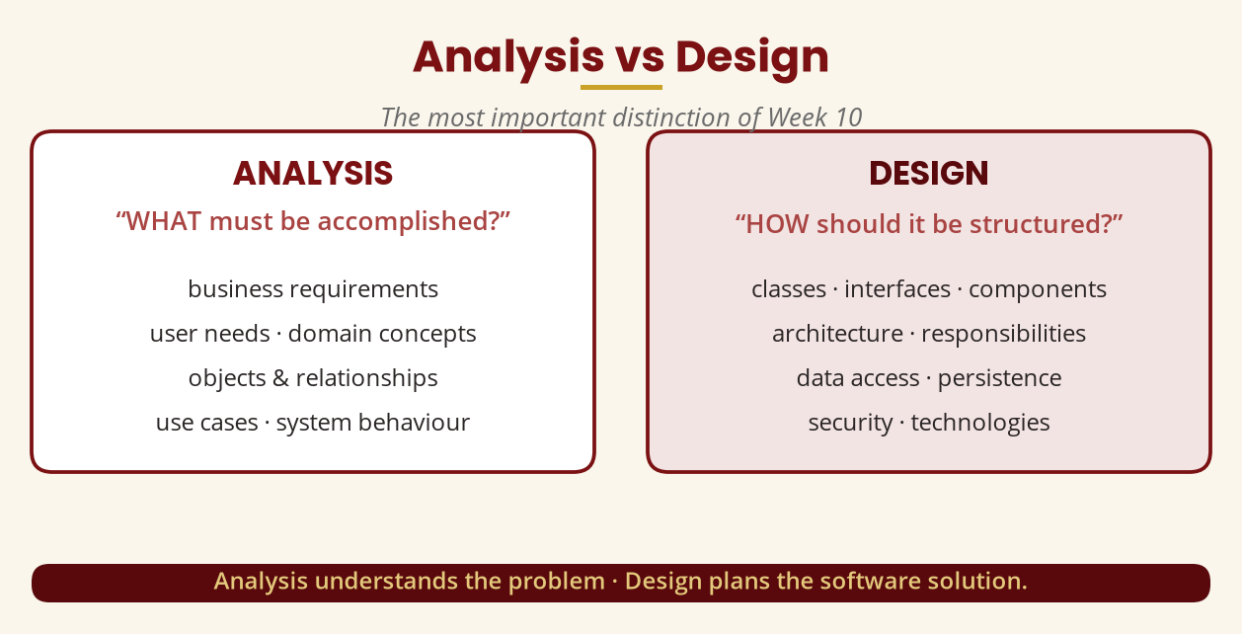

Figure 3 — Analysis (WHAT) vs Design (HOW)

6. Worked Example — "Register for a Course"

Given the requirement *"A student should be able to register for a course."*, **analysis** identifies the concepts **Student, Course, Registration**. **Design** then asks how the software will make this happen and introduces **RegistrationService, CourseRepository, StudentRepository** and the flow Student RegistrationService CourseRepository Database. Now we are making implementation-oriented decisions.

7. Analysis Model vs Design Model

The **analysis model** describes the logical structure of the application without focusing on how it will be implemented; the **design model** builds on it and adds implementation-oriented detail — design classes, interfaces, packages and architectural elements.

Analysis Model vs Design Model

The analysis model is the foundation of the design model

	ANALYSIS MODEL	DESIGN MODEL
Focus	Understanding the problem	Solving the problem
Answers	What the system needs	How it will be built
Nature	More conceptual	More technical
Orientation	Domain-oriented	Implementation-oriented
Technology	Less technology-specific	More technology-specific

The analysis model describes logical structure · the design model adds implementation detail.

Figure 4 — Analysis model vs design model

PART B — WHY DO WE NEED DESIGN?

8. Why Not Just Start Coding?

A lecturer says *"Build a university management system"* and the programmer immediately opens the editor and writes `student.java`. Three months later that file has 15,000 lines containing login, payments, course registration, reports, database queries, email, SMS, authentication and results — and changing one feature breaks five others. That is exactly why design matters. **Good design makes systems easier to understand, implement, test, maintain, modify, extend, reuse, debug and scale.**

9. Design Goals

A good object-oriented design should generally aim for eight goals:

- **Correctness** — the design satisfies the requirements.
- **Maintainability** — developers can modify it without destroying unrelated functionality.
- **Reusability** — useful components can be reused elsewhere.
- **Flexibility** — the system can adapt to changing requirements.
- **Understandability** — developers can understand how it works.
- **Testability** — individual parts can be tested effectively.
- **Scalability** — the system can accommodate increased demand where required.
- **Security** — security requirements are incorporated into the design.

Figure 5 — Eight design goals

10. Poor vs Good Design

A poor design puts everything — login, registration, fees, email, reports, database — into one huge class. A better design splits them into focused services such as **AuthenticationService, RegistrationService, PaymentService, EmailService, ReportService, ResultService**.

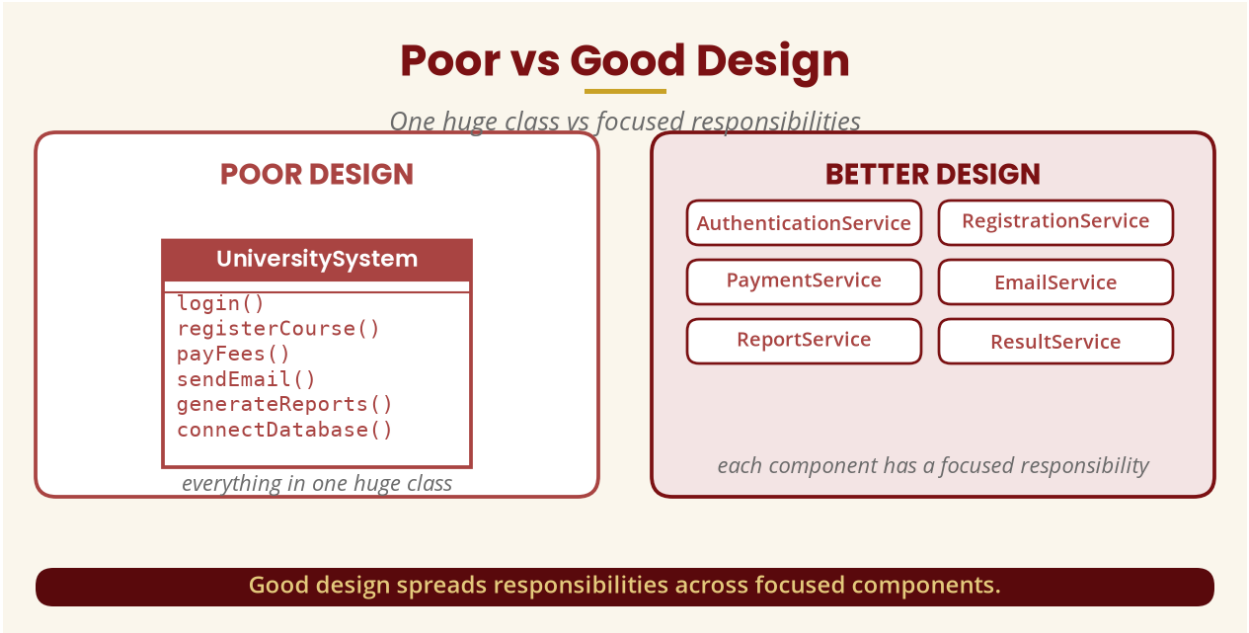

Figure 6 — Poor vs good design

PART D — RESPONSIBILITY

11. What Is Responsibility?

A **responsibility** is something an object or class is responsible for **knowing** or **doing**. A `Student` should know its `studentID`, `name`, `programme` and should do `registerCourse()` and `dropCourse()`. A `Course` should know its `courseCode`, `title`, `credits`, `capacity` and should do `hasSpace()`.

Responsibility assignment is one of the central design questions: *"Which object should be responsible for this operation?"* To calculate total fees, should it be `Student.calculateFees()`, `FeeAccount.calculateTotal()` or `PaymentService.calculateFees()`? The designer chooses based on the responsibilities and relationships of the objects.

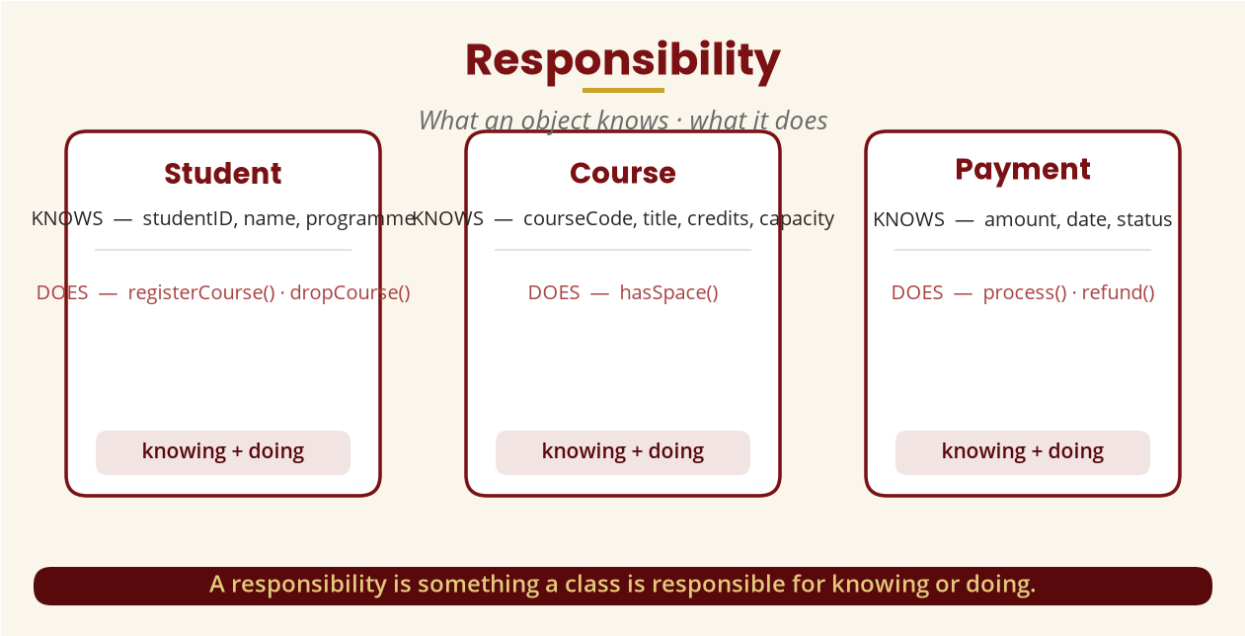

Figure 7 — Responsibility: knowing and doing

PART E & F — COHESION AND COUPLING

12. Cohesion

Cohesion describes how closely related the responsibilities inside a class or module are. **High cohesion** means a class focuses on closely related responsibilities — a `Course` with `courseCode`, `title`, `credits`, `capacity`, `hasSpace()`, `addStudent()`, `removeStudent()` is cohesive. **Low cohesion** means a class does too many unrelated things. Ask: *"Does everything inside this class belong together?"*

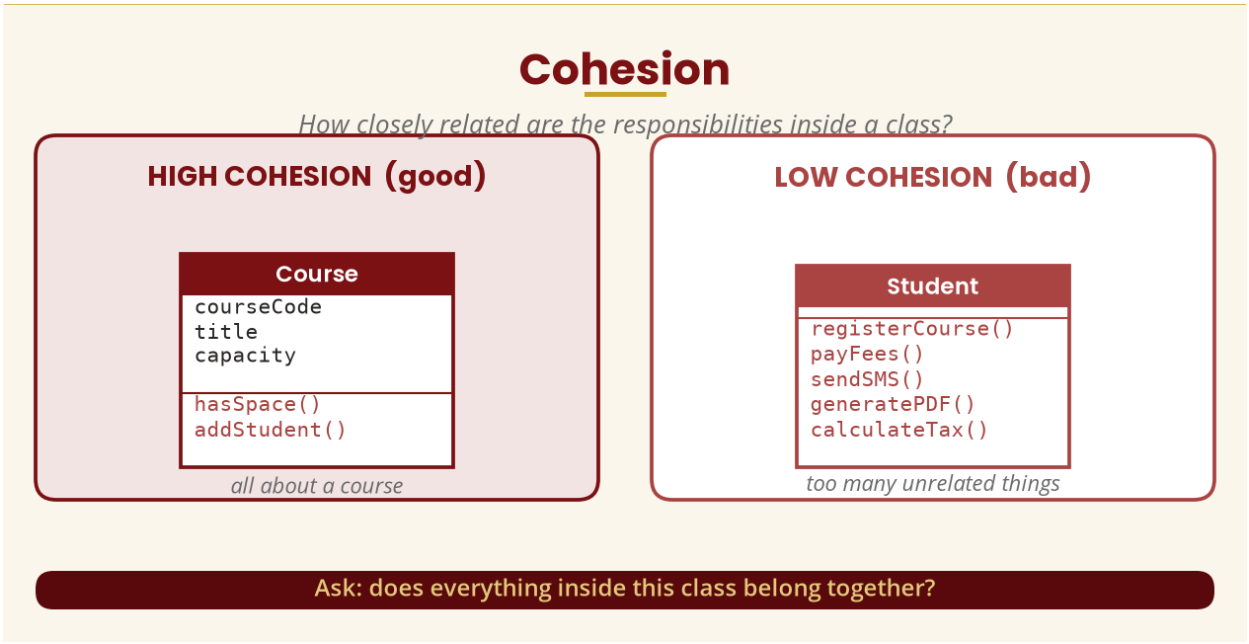

Figure 8 — High vs low cohesion

13. Coupling

Coupling describes how strongly one class/module depends on others. **High coupling** means every class directly depends on every other class — changing one part becomes dangerous. **Low coupling** means components communicate through clearly defined interfaces, which makes substitution and testing easier. Generally good OO design aims for **low coupling**.

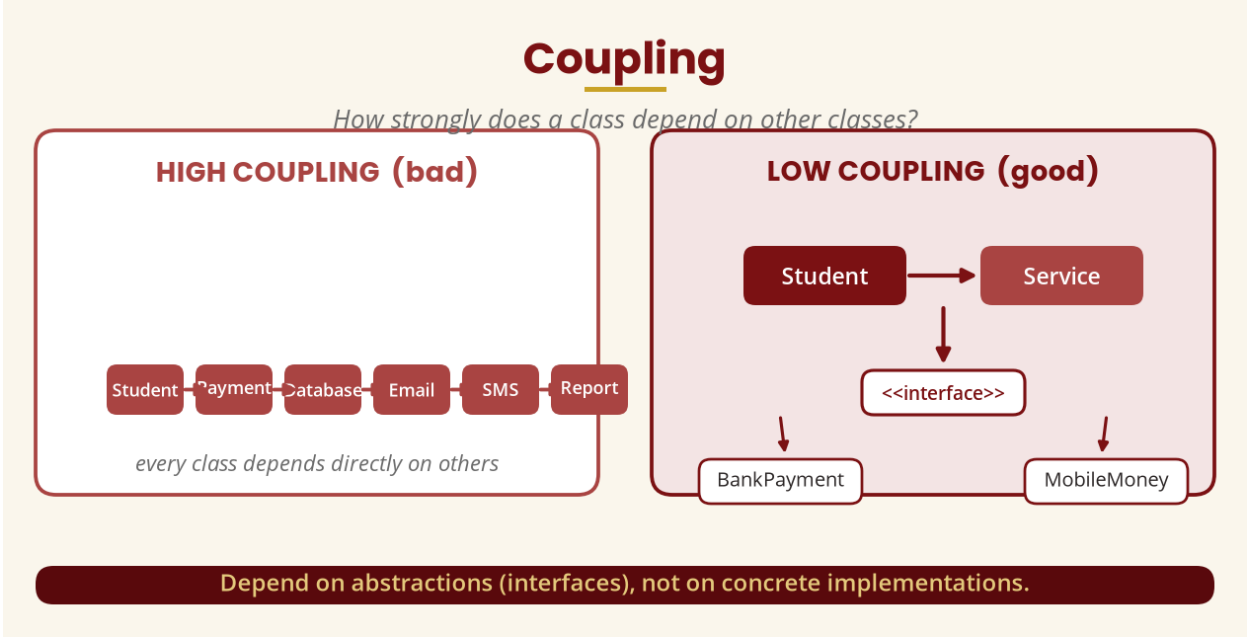

Figure 9 — High vs low coupling

14. Cohesion vs Coupling — Do Not Confuse Them

Cohesion looks inside a component (how closely related are its responsibilities?); **coupling looks between components** (how strongly do they depend on each other?). Therefore:

GOOD DESIGN = HIGH COHESION + LOW COUPLING

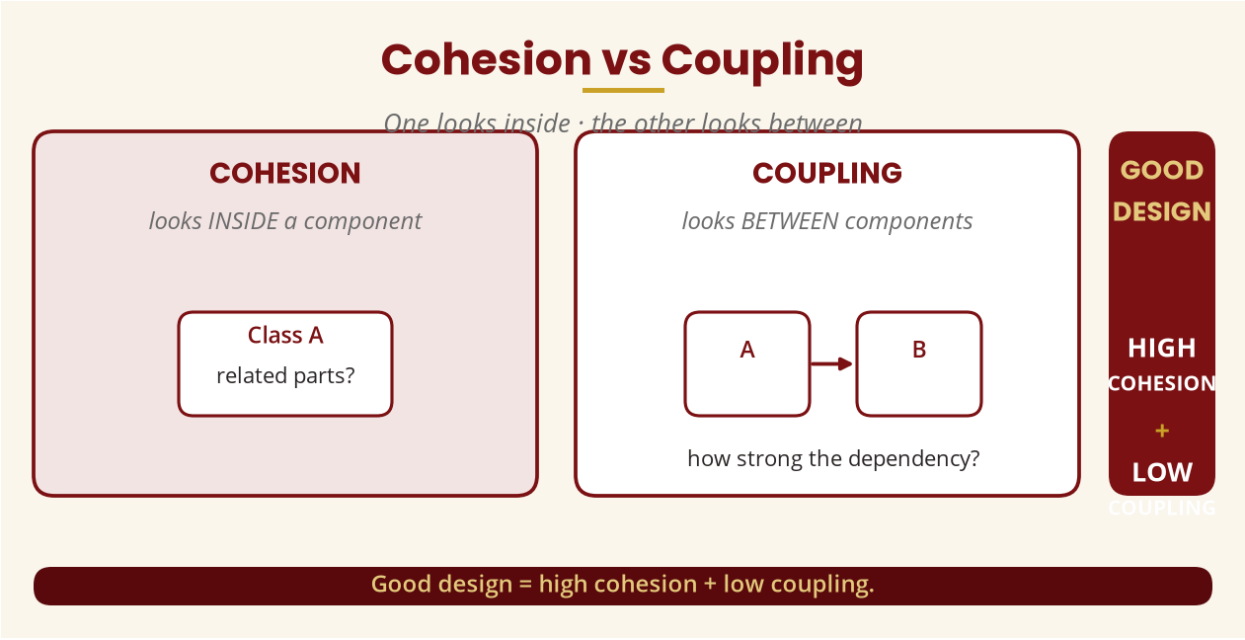

Figure 10 — Cohesion vs coupling

PART G & H — ABSTRACTION AND ENCAPSULATION

15. Abstraction

Abstraction means focusing on the important characteristics of something while hiding unnecessary details. Using an ATM you insert a card, enter a PIN, select withdrawal, enter an amount and receive money — you do not need to know how the bank's database, encryption, communication or cash dispenser work. In OO design, a `PaymentProcessor.processPayment()` hides the internal detail from the rest of the application.

16. Encapsulation

Encapsulation means keeping data and the operations that manage that data together while controlling access to the internal state. A `BankAccount` has a private `balance` and public `deposit()`, `withdraw()` and `getBalance()` — the balance is changed through `account.deposit(500)`, never `account.balance = 500`. This protects the object's state.

Abstraction vs encapsulation: abstraction focuses on *what the user should see*; encapsulation focuses on *what the object should hide/protect internally*.

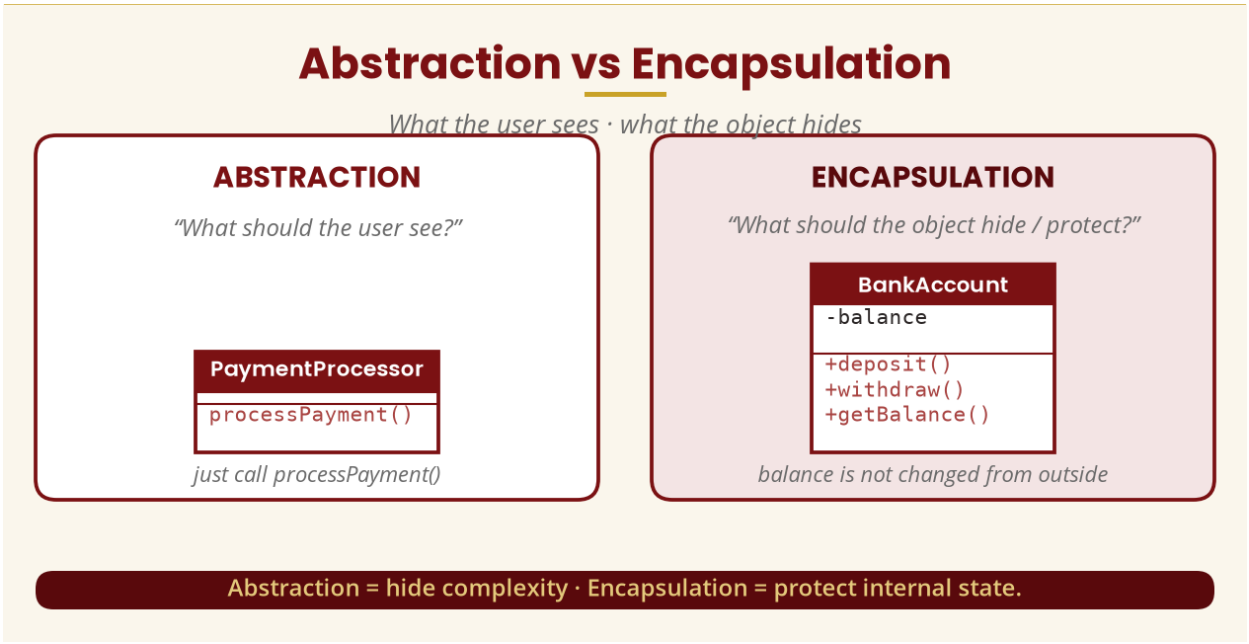

Figure 11 — Abstraction vs encapsulation

PART I & J — SEPARATION OF CONCERNS AND ARCHITECTURE

17. Separation of Concerns

Separation of concerns means dividing a system so that different concerns are handled by appropriate parts — Presentation Business Logic Data Access Database — rather than mixing SQL, password hashing, HTML and database connections into one `LoginScreen`.

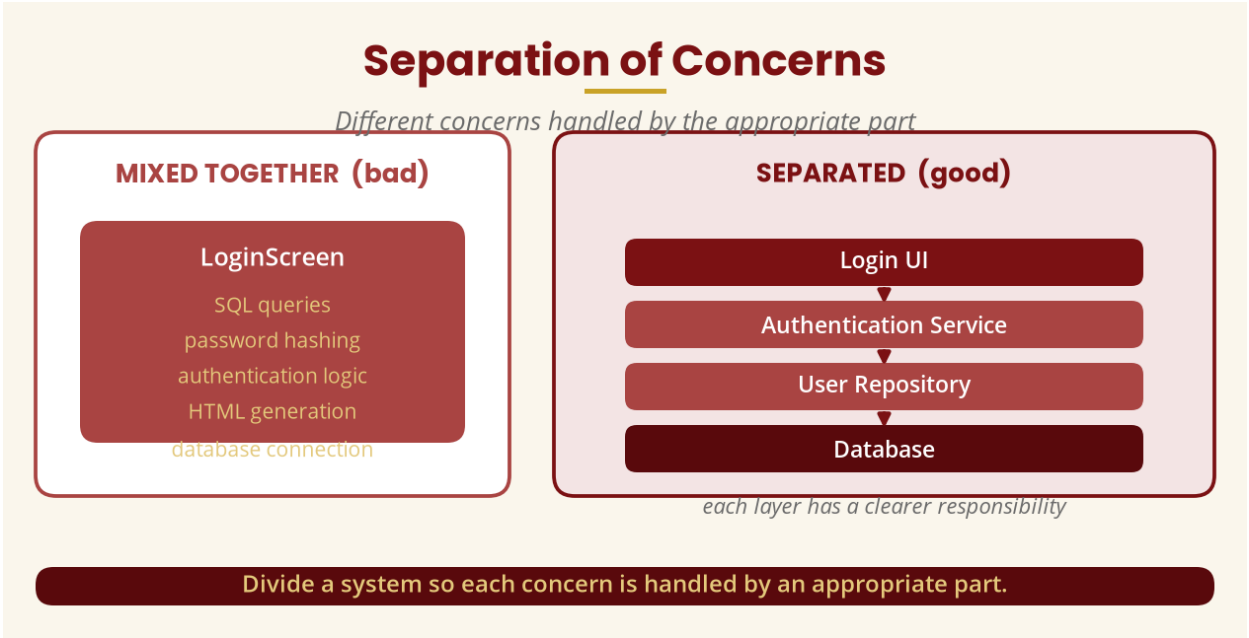

Figure 12 — Separation of concerns

18. Design Architecture

Software architecture is the high-level organisation of a software system — its major components, layers, interfaces, dependencies, communication mechanisms and deployment decisions. A common conceptual structure is the **three-layer architecture**: Presentation (Web/Mobile/Desktop UI), Business Logic (rules/services), and Data (repositories/database). Example — a student clicks **Register Course**: the presentation forwards to business logic, which checks that the student is active, the course is available, prerequisites are met and there is space, and the data layer retrieves and updates the records.

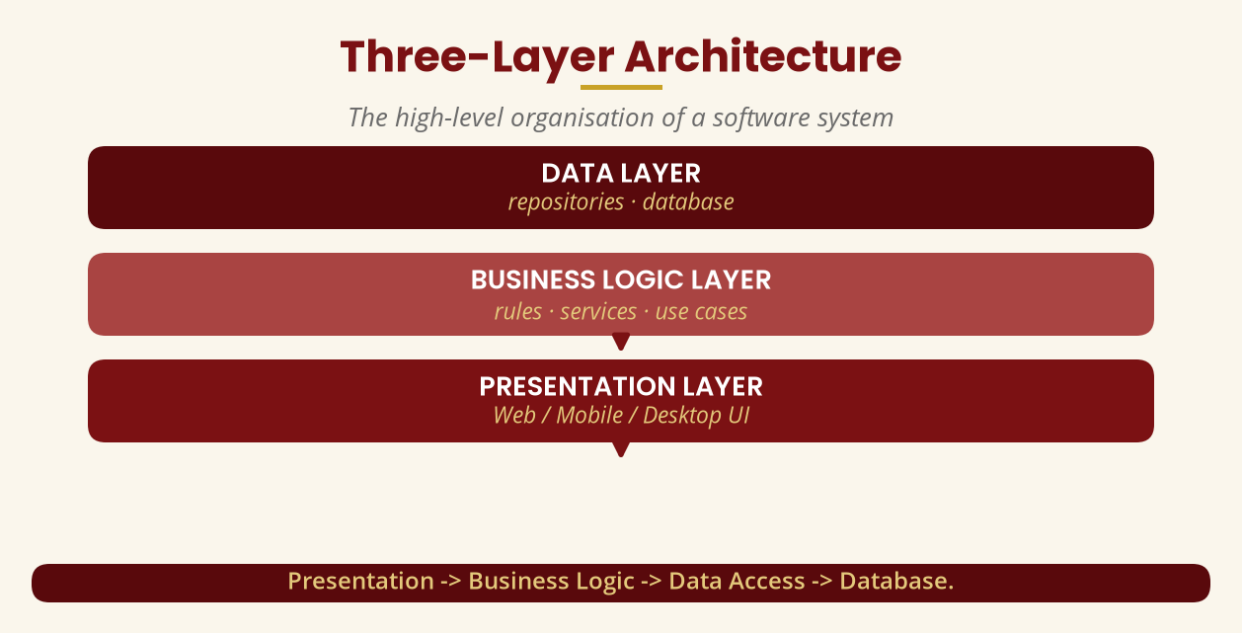

Figure 13 — Three-layer architecture

PART K — DESIGN REFINEMENT

19. Refinement

Refinement means taking a high-level model and progressively adding detail. The analysis class `Student` becomes a design class with typed attributes (`studentId: String`, `name: String`, `email: String`, `status: StudentStatus`) and typed operations (`registerCourse()`, `dropCourse()`, `getRegisteredCourses()`), and finally implementation in `Student.java`, `Student.cs` or `Student.ts`. The path is: Real-world problem Analysis model Design model Implementation Running software.

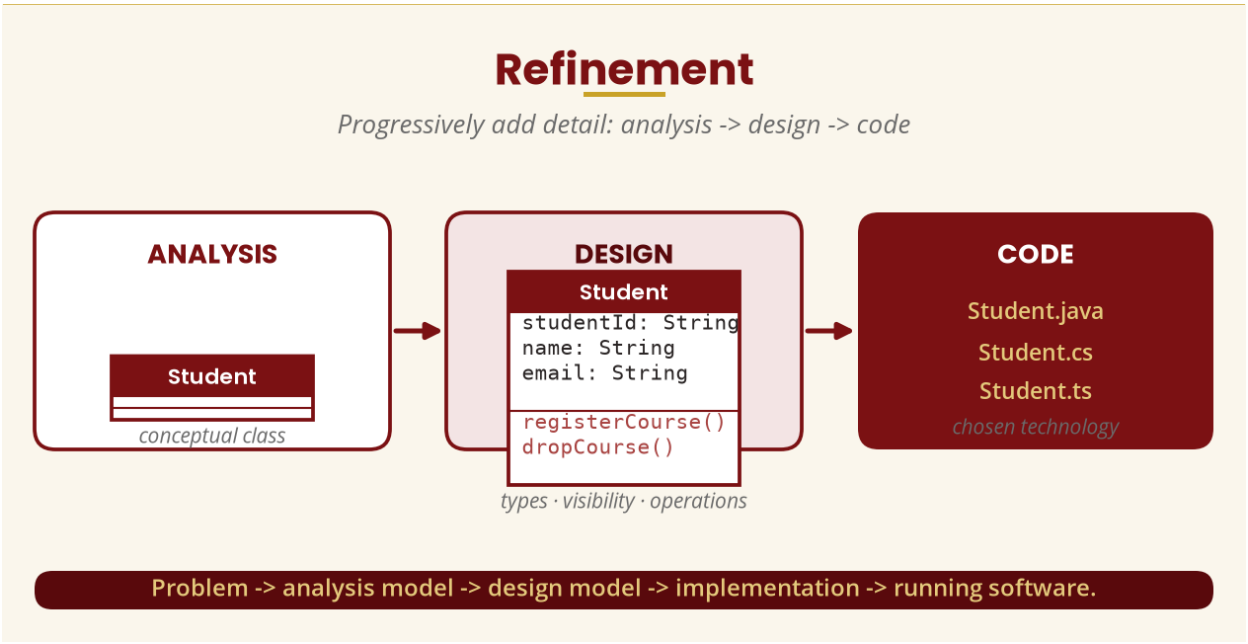

Figure 14 — Refinement: analysis design code

PART L & M — COMBINING THE THREE MODELS (the core of Week 10)

20. The Three OOAD Models

Object Model — describes the **structure**: what things/classes exist (Student, Course, Registration, Payment). **Dynamic Model** — describes how objects change over **time** and respond to events (Registration: Requested Pending Confirmed, or Pending Rejected). **Behavioural Model** — describes **what the system does** and how actors/objects interact (Student Register Course Validate Registration Confirm Registration).

Why do we need all three? Knowing the parts of a car (Wheel, Engine, Brake, Steering) is not enough — we also need to know what happens when the driver presses the brake, and how the brake, wheel and engine systems interact. In software, **structure + behaviour + time/state must fit together**.

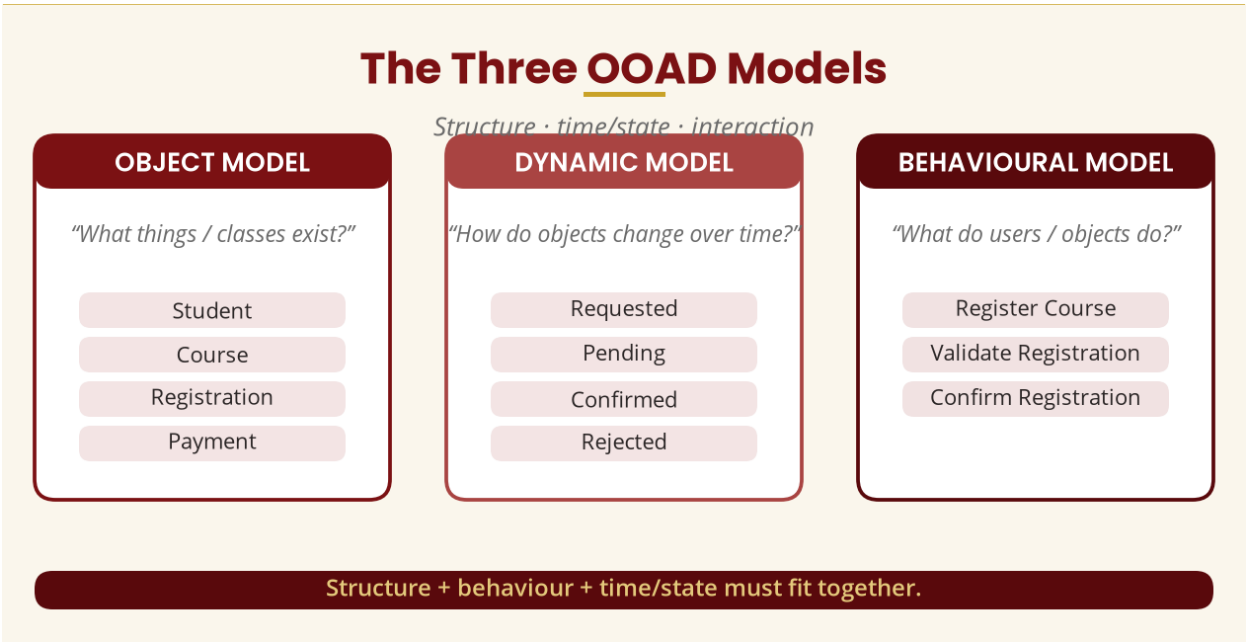

Figure 15 — The three OOAD models

21. Case Study — Online Course Registration (7 Steps)

Requirement: *"A student should be able to register for an available course."* **(1)** Behavioural model — use case **Register Course**. **(2)** Identify objects: Student, Course, Registration. **(3)** Object model — the classes with their attributes. **(4)** Relationships — Student 1 — 0..* Registration — Course. **(5)** Dynamic model — Registration states Requested Validated Confirmed (or Rejected). **(6)** Sequence — Student RegistrationService Course Registration Student. **(7)** Design — Student, Course, Registration plus RegistrationService, CourseRepository and RegistrationRepository.

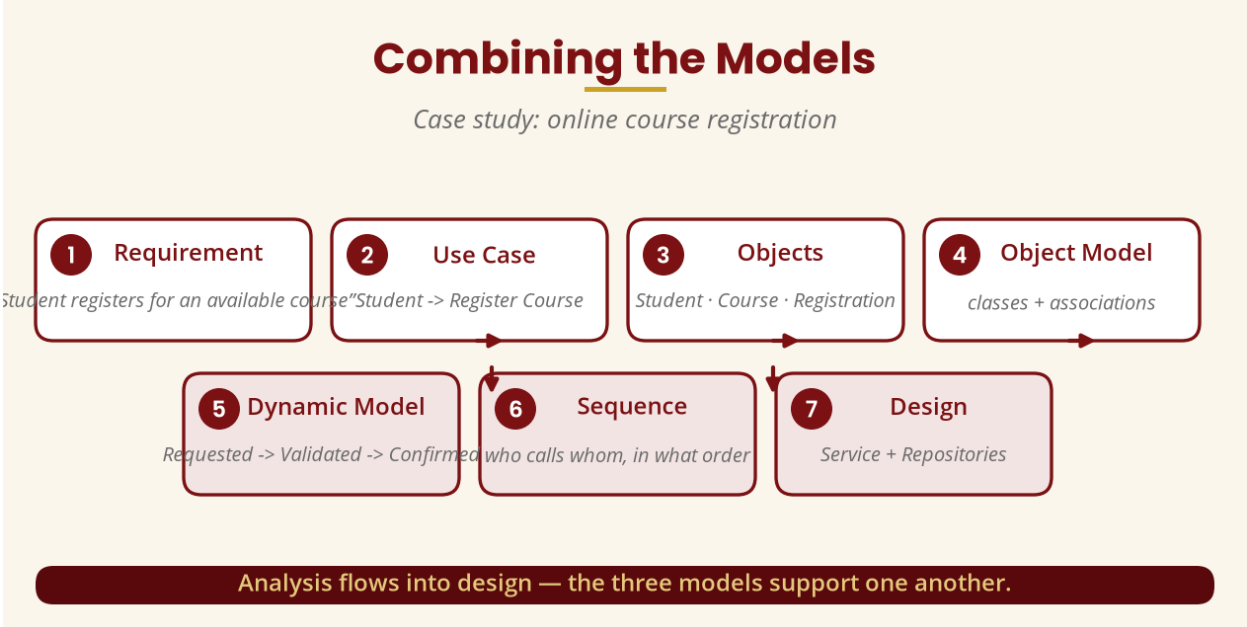

Figure 16 — Combining the models

PART N — MODEL CONSISTENCY AND TRACEABILITY

22. Model Consistency

The different models should not contradict each other. If the class model gives `Course` only `courseCode, title, capacity` but the sequence diagram calls `Course.checkPrerequisite()`, the operation is missing — an inconsistency. Likewise, if the use case actor is **Student** but the sequence diagram shows `Lecturer → registerCourse()`, the models disagree and must be reconciled.

23. Traceability

Traceability means being able to follow a requirement through the development artefacts: Requirement Use Case Activity diagram Sequence diagram Class model Design classes Code Test case. If the requirement changes, developers can identify what models and code may be affected — this is extremely valuable.

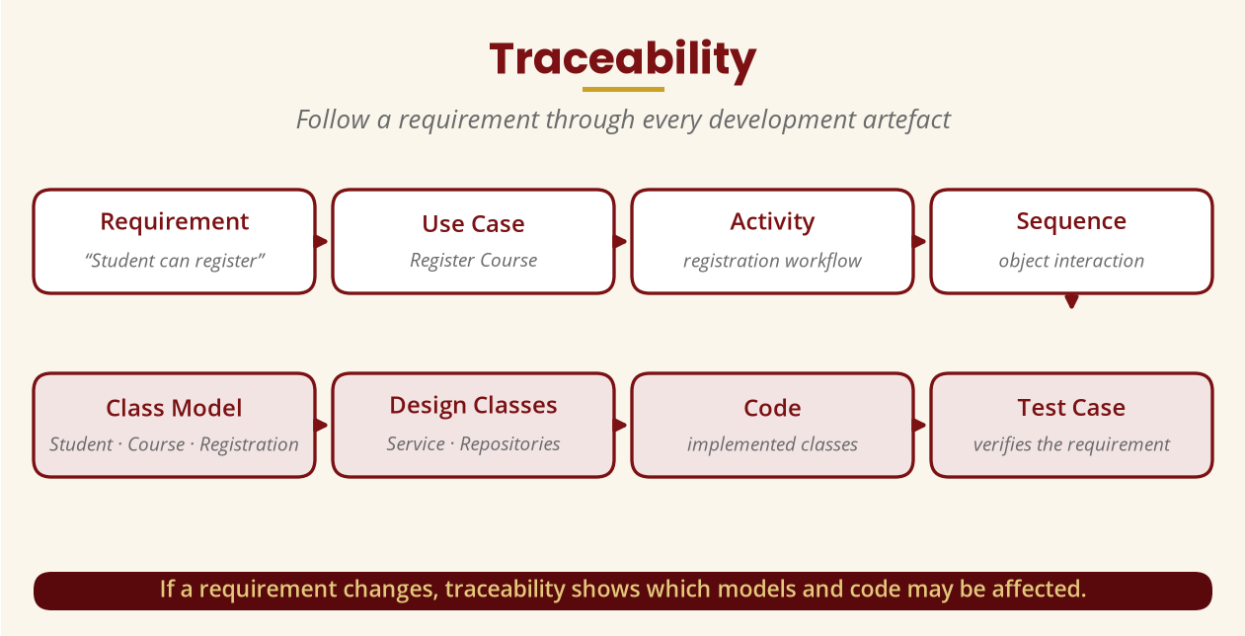

Figure 17 — Traceability

PART O–Q — SEQUENCE DIAGRAMS, DESIGN CLASSES AND INTERFACES

24. Sequence and Class Diagrams in Design

Sequence diagrams show the order of interactions between objects — Student RegistrationService `register()` Course `checkAvailability()` — telling us far more than simply that Student, Course and Registration exist. **Design class diagrams** enrich the analysis classes with **visibility, attribute types, parameter types, return types and operations** (e.g. `+registerCourse(c: Course)`, `+getCourses(): List<Course>`).

25. Interfaces

An **interface** defines a set of operations that a class/component agrees to provide — for example `<<interface>> PaymentProcessor + processPayment()`, implemented by both `BankPayment` and `MobileMoneyPayment`. When the university later adds mobile money, the application can depend on the abstraction and avoid unnecessary coupling.

PART R & S — DESIGN PATTERNS AND SOLID (outline)

26. Design Patterns

A **design pattern** is a reusable solution structure for a recurring design problem — e.g. Factory, Observer, Strategy, Adapter, Singleton. Example: the **Strategy pattern** conceptualises `PaymentStrategy` with Bank, Mobile and Card implementations instead of a chain of `if paymentType == …` branches, making the design more flexible.

27. SOLID Principles (outline)

S — Single Responsibility; **O** — Open/Closed; **L** — Liskov Substitution; **I** — Interface Segregation; **D** — Dependency Inversion. These are **guidelines**, not magical rules, and relate closely to cohesion, coupling, abstraction and dependency management. The Single Responsibility Principle says a class should have a focused responsibility rather than accumulating unrelated ones; the Open/Closed Principle says entities should be open for extension but closed for modification.

PART T — THE COMPLETE OO DESIGN PROCESS

Requirement Use-case model Behavioural model interactions Object model (Student — Registration — Course) Dynamic model (Requested Validated Confirmed) Design model (RegistrationService, CourseRepository, RegistrationRepository) Architecture (Presentation / Business Logic / Data Access) Implementation Code. That is the central lesson of Week 10.

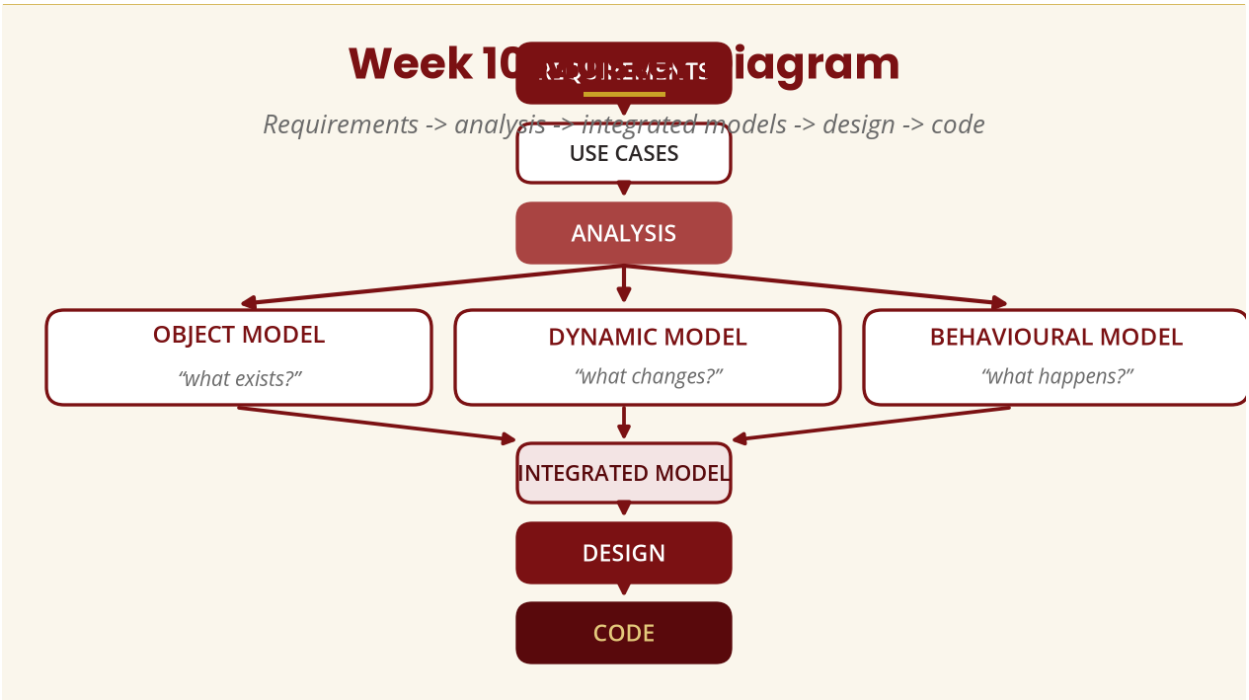

Figure 18 — Week 10 master diagram

28. Practical Class

Working individually or in groups, design an online student registration system from the requirement: *"A student logs into the university system, searches for a course, selects the course, and registers if all conditions are satisfied."*

- **Task 1** — Identify classes (Student, Course, Registration…) and justify each.
- **Task 2** — Add responsibilities (Student: login(), registerCourse(), dropCourse(); Course: checkAvailability(), checkPrerequisite(); Registration: create(), confirm(), cancel()).
- **Task 3** — Draw a class diagram (classes, attributes, operations, associations, multiplicities).
- **Task 4** — Draw a sequence diagram (Student RegistrationService Course Registration).
- **Task 5** — Draw a state machine for Registration (Requested Validated Confirmed; alternative Rejected).
- **Task 6** — Critique a class that contains login(), registerCourse(), payFees(), sendSMS(), generateReport(), connectDatabase(), calculateResults() and uploadAssignment(): is cohesion high or low, why, and how could it be redesigned?

29. Tutorial Questions

- Define Object-Oriented Design.
- Differentiate Object-Oriented Analysis from Object-Oriented Design, with an example.
- Why should a team not immediately begin coding after receiving a requirement?
- Explain cohesion and coupling, and why good design seeks high cohesion and low coupling.
- Differentiate abstraction and encapsulation using a BankAccount example.
- Explain the Object, Dynamic and Behavioural models.

- Explain why the three models must be consistent.
- For “Customer withdraws money”, identify possible classes, responsibilities, interactions and states.

30. Common Misunderstandings

- **“Analysis is just drawing diagrams.”** — Analysis is understanding requirements, domain, users, objects and behaviour; UML is only the representation.
- **“Design means coding.”** — Coding is implementation; design determines what the implementation should look like.
- **“A class diagram is enough.”** — It mainly shows structure; sequence, state, activity, component and deployment diagrams may also be needed.
- **“More classes always mean better design.”** — The goal is an appropriate structure, not the maximum number of classes.
- **“Inheritance should always be used.”** — Use it only when a genuine IS-A relationship exists (a Student is NOT a Course).

31. Week 10 Summary and Key Terms

Concept	Student-friendly meaning
Software design	Deciding how the software will be structured.
OO design	Designing around objects/classes and their responsibilities.
Analysis vs design	Understanding the problem vs planning the solution.
Responsibility	What a class knows or does.
Cohesion	How closely related responsibilities inside a component are.
Coupling	How strongly components depend on each other.
Abstraction	Showing important things, hiding unnecessary detail.
Encapsulation	Protecting internal data / implementation.
Interface	Contract of operations a component provides.
Architecture	High-level organisation of the system.
Refinement	Adding detail to an existing model.
Traceability	Following a requirement through models to code.
Object / Dynamic / Behavioural model	Structure / states over time / interactions.
Design pattern	Reusable solution structure for a recurring problem.

THE BIG IDEA

Analysis tells us what the system needs; design determines how the software will be organised to satisfy those needs. The three models are not competing — they complement one another: Object Model (what exists) + Dynamic Model (what changes) + Behavioural Model (what happens) integrated understanding object-oriented design implementable software.

32. Examination-Style Questions

Essay (25 marks). Explain the role of Object-Oriented Design in software development. Distinguish OO analysis from OO design, and discuss how the Object, Dynamic and Behavioural models can be combined to produce a design suitable for implementation.

Scenario (20 marks). A university wants an online student registration system in which students log in, search for courses, register, drop courses and view registered courses. (a) Identify five classes. (b) Assign at least two responsibilities to each. (c) Draw a class diagram. (d) Draw a sequence diagram for course registration. (e) Draw a state diagram for the Registration

object. (f) Explain how the three models support one another.

33. References and Further Reading

Authoritative / current online sources

- Object Management Group (OMG). *Unified Modeling Language (UML), Version 2.5.1.* OMG. https://www.omg.org/uml/
- IBM. *Capturing solution architecture in a design model.* IBM documentation — the design model builds on the analysis model with design classes, interfaces, packages, sequence and state diagrams, components and deployment views. https://www.ibm.com/docs/en/dma
- IBM. *Capturing application domain in an analysis model.* IBM documentation — the analysis model as the logical foundation for design. https://www.ibm.com/docs/en/dma
- Hu, C. (2023). *An Introduction to Software Design: Concepts, Principles, Methodologies, and Techniques.* Springer. https://link.springer.com/book/10.1007/978-3-031-28311-6

Prescribed and recommended texts

- Mach, E. (2019). *Object Oriented Analysis & Design Cookbook: Introduction to Practical System Modeling.*
- Reddy, V., & Korra, S. (2018). *Object-Oriented Analysis and Design Using UML.*
- The Art of Service. (2021). *Object Oriented Analysis and Design: A Complete Guide.*
- Wixom, B., & Dennis, A. (2018). *Systems Analysis and Design.*
- Blokdyk, G. (2021). *Object-Oriented Analysis and Design Standard Requirements.*
- Wixom, B., & Dennis, A. (2020). *Systems Analysis and Design: An Object-Oriented Approach with UML.*